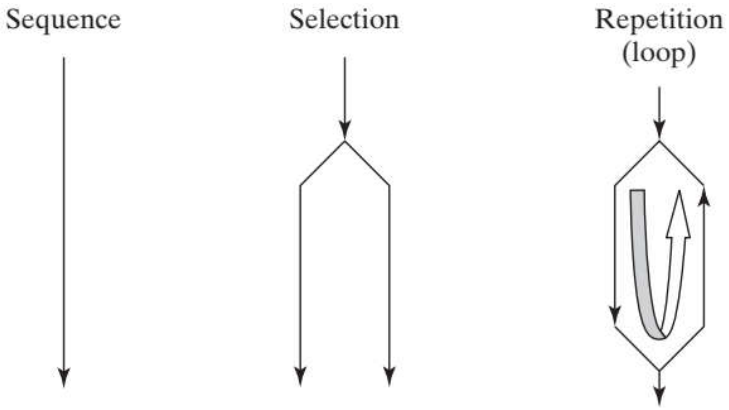
PROGRAMMING STRUCTURES (FLOW CONTROL)

Programming Structures

- Sections of a code or program (script or function) can be categorized as;
  - sequences,
  - selection (branching) structures, and
  - repetition structures



H. Moore, Matlab for Engineers.

Relational and Logical Operators

- The selection and repetition structures depend on relational and logical operators.
- Relational operators compare two numbers produce a true or false result.
- Logical operators compare two logical values. They also combine comparison statements.
- For matrices of the equal size, the following operators are available in MATLAB/Octave;

Relational Operator	Interpretation
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to
==	equal to
~=	not equal to

Logical Operator	Interpretation
&	and
~	not
|	or
xor	exclusive or

Relational and Logical Operators

Relational Operators.

Operation	Result
3 < 4	true (1)
3 <= 4	true (1)
3 == 4	false (0)
3 > 4	false (0)
4 <= 4	true (1)
'A' < 'B'	true (1)

```
>> x = [ 1, 2, 3, 4, 5];
>> y = [-2, 0, 2, 4, 6];
>> x<y
>> ans =
      0   0   0   0   1
```

Logical Operators.

```
>> x = [ 1, 2, 3, 4, 5];
>> y = [-2, 0, 2, 4, 6];
>> z = [ 8, 8, 8, 8, 8];
>> z>x & z>y
>> ans =
         1   1   1   1   1

>> x>y | x>z
>> ans =
         1   1   1   0   0
```

Selection (Branching Structures)

- Enable us to run or specific blocks (sections) of code, while skipping other sections.

- Variations include the
  - if construct,
  - switch construct,
  - try/catch constructs.

The `if` structure

- Takes the form

```
if control_expr_1
   Statement 1
   Statement 2          } Block 1
   ...
elseif control_expr_2
   Statement 1
   Statement 2          } Block 2
   ...
else
   Statement 1
   Statement 2          } Block 3
   ...
end
```

- For instance, say we have a script saved as sc_GG whose contents are as follows;

```
if G<50
    disp('G is a small value equal to:')
    disp(G);
end
```

- If in the command prompt, we should run;

```
>> G = 25
>> sc_GG
```

- We should get;

```
G is a small value equal to:
25
```

The `if` structure

➢E.g. Let's say we have a script saved as iftrial whose contents are as follows;

```
%prompt user for input value
x = input('Enter a value of x greater than 0: ');
%compare x to 0. if larger, computer the log of x and assign it to y
if x >0
   y = log(x)
%if x is less than 0, display a message
else
   disp('The input to the log function must be positive')
end
```


The `if` structure

➢Suppose you are to write a program which reads in a numerical grade and assigns a letter grade to it according to the following table:

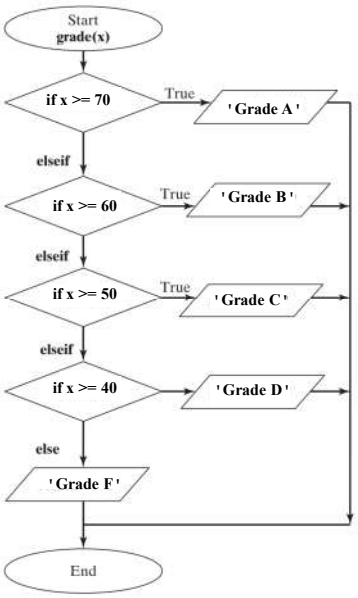
$70 \leq$ grade	A
$60 \leq$ grade < 70	B
$50 \leq$ grade < 60	C
$40 \leq$ grade < 50	D
$0 \leq$ grade < 40	F

```
if grade >= 70.0
        disp('Grade is A.');
elseif grade >= 60.0
        disp('Grade is B.');
elseif grade >= 50.0
        disp('Grade is C.');
elseif grade >= 40
        disp('Grade is D.');
else
        disp('Grade is F.');
end
```

Flow charts and Pseudocodes

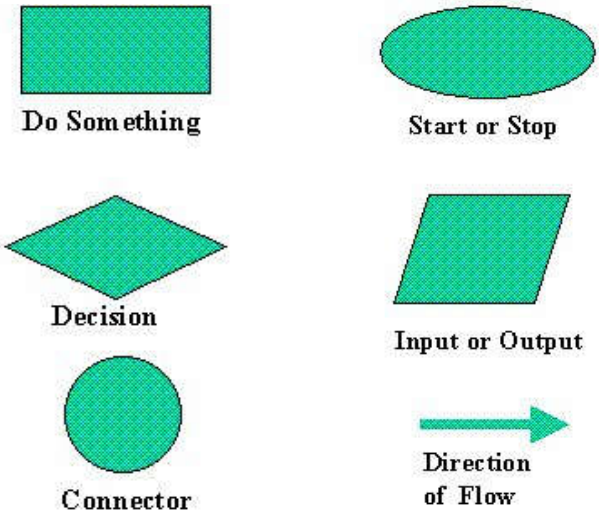
- Help to describe the algorithm that you intend to implement.

Flow charts




```
elseif grade >= 70.0
       disp('Grade is A.');
elseif grade >= 60.0
       disp('Grade is B.');
elseif grade >= 50.0
       disp('Grade is C.');
elseif grade >= 40
       disp('Grade is D.');
else
       disp('Grade is F.');
end
```

Flow chart Symbols often used in Computer Programming



Pseudocodes

- Pseudocode is a hybrid mixture of MATLAB and English that describes the algorithm that you intend to implement.

```
% This code reads in a numerical grade and assigns an appropriate letter grade to it.
% Assuming the score has already been entered
% Check if score is greater than 70 and assign grade A
    if grade >= 70.0
        disp('Grade is A.');
% Score is less than 70, so check if score is greater than 60 and assign grade B
    elseif grade >= 60.0
        disp('Grade is B.');
% Score is less than 60, so check if score is greater than 50 and assign grade C
    elseif grade >= 50.0
        disp('Grade is C.');
% Score is less than 50, so check if score is greater than 40 and assign grade D
    elseif grade >= 40
        disp('Grade is D.');
% Score is less than 40, so assign grade F
    else
        disp('Grade is F.');
    end
```

The `switch/case` structure

- Similar to `if else if` structure.
- Code branches depending on if the value of a **switch** variable matches the value of the **case** variable. Switch is more often used for string criteria.
- Takes the form

```
switch (switch_expr)
case case_expr_1
    Statement 1
    Statement 2          } Block 1
    ...
case case_expr_2
    Statement 1
    Statement 2          } Block 2
    ...
...
otherwise
    Statement 1
    Statement 2          } Block n
    ...
end
```


The `switch/case` structure

- E.g.
- Suppose you want to create a function that tells the user what the Intercity STC busfare is from Accra one of three different cities:

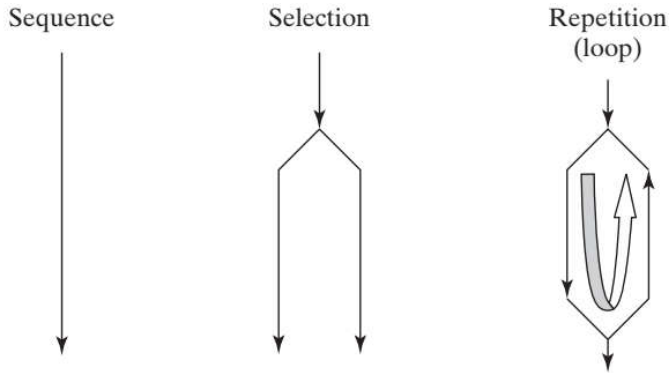
```
city = input('Enter the name of a city in single quotes: ')
switch city
   case 'Kumasi'
      disp('GH₵45')
   case 'Takoradi'
      disp('GH₵ 47')
   case 'Tamale'
      disp('GH₵135')
   otherwise
      disp('Kindly call our office')
end
```

The `switch/case` structure

➢The previous code could have also been

```
city = input('Enter the name of a city: ','s')
switch city
   case 'Kumasi'
      disp('GH₵45')
   case 'Takoradi'
      disp('GH₵ 47')
   case 'Tamale'
      disp('GH₵135')
   otherwise
      disp('Kindly call our office')
end
```


Repetition Structures (Looping)



➢Blocks (sections) of code, are repeated when specified conditions are met.

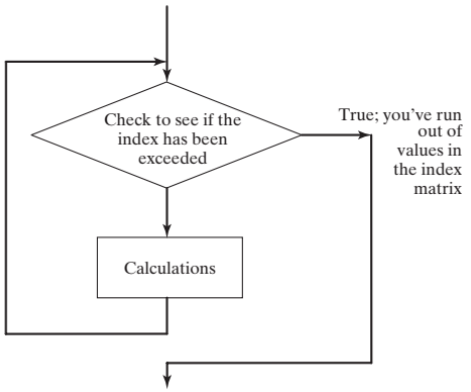
➢ Variations are the `while` and `for` structures.

The for loop

- Executes a block of statements a certain number of times.
- The for loop has the form;

```
for index = expr
   ...
   ...
   ...
end
```

Body



- The loop is run for each element in the index.

The for loop

- The code below;

```
for k = [1,3,7]
   k
end
```

- Should return

```
k =
  1
k =
  3
k =
  7
```

- The code below;

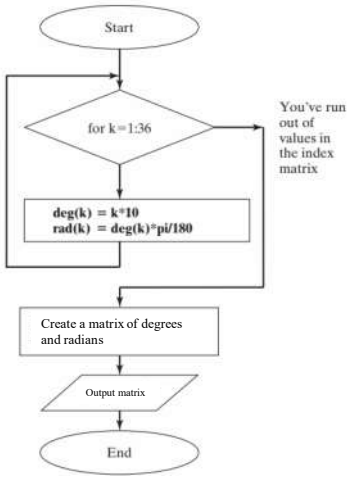
```
for
  k = 1:3
  a = 5^k
End
```

- Should return

```
k =
  5
k =
  25
k =
  125
```

The for loop

- Write a code that converts angle values ranging from 0 to 360 degrees (in increments of 10) to radians, from 0 to 360 degrees.




```
%Create a matrix of degrees on row1 and
%radians on row 2
%Use a for loop for the calculations

for k=1:36
   deg(k) = k*10;
   rad(k)=deg(k)*pi/180;
end

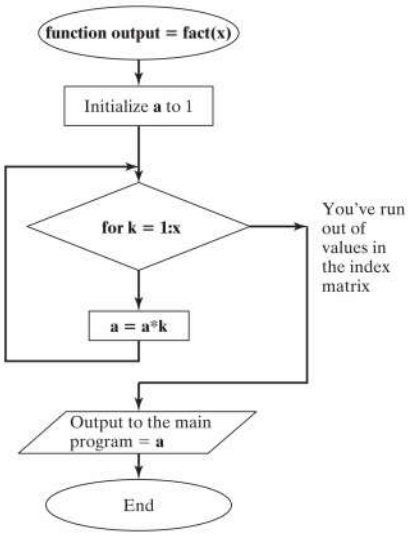
%Create a matrix of results

t = [deg;rad]
%display the matrix

disp(t)
```


The for loop

- Write a function that takes a number and returns its factorial.




```
function output = fact(x)
%This function accepts a scalar input
%and calculates its factorial

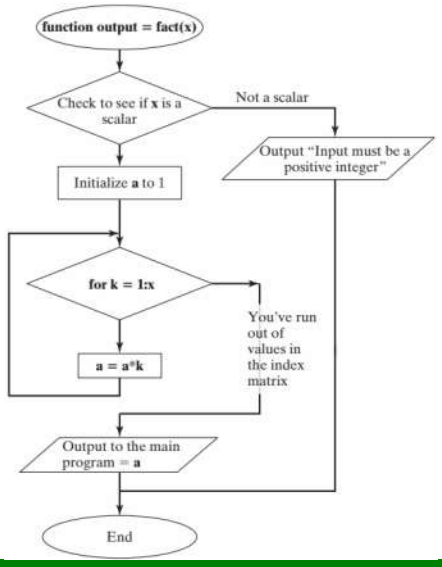
% initialize a
a = 1;

%Use a loop to calculate the factorial
for k = 1:x
  a = a*k;
end

output = a;
```

The `for` loop

- for loops can be nested within branching structures and vice versa.




```
function output = fact(x)
%This function accepts a scalar input and
%calculates its factorial

%Check to confirm that x is a positive integer
if(x<0)
   output = 'Input must be a positive integer'
else
    % initialize a
    a = 1;
    %Use a loop to calculate the factorial
    for k = 1:x
      a = a*k;
    end
    output = a
end
```


"Vectorizing" code

- Vectorize your code where possible to avoid `for` loops.

```
%Create a matrix of degrees on row1
%and radians on row 2
%Use a for loop for the calculations

for k=1:36
   deg(k) = k*10;
   rad(k)=deg(k)*pi/180;
end

%Create a matrix of results

t = [deg;rad]
%display the matrix

disp(t)
```

=

```
%Create a matrix of degrees on row1
%and radians on row 2
%create row matrix of degrees
deg = 10:10:360

%convert to radians
rad=deg*pi/180;

%Create a matrix of results
t = [deg;rad]

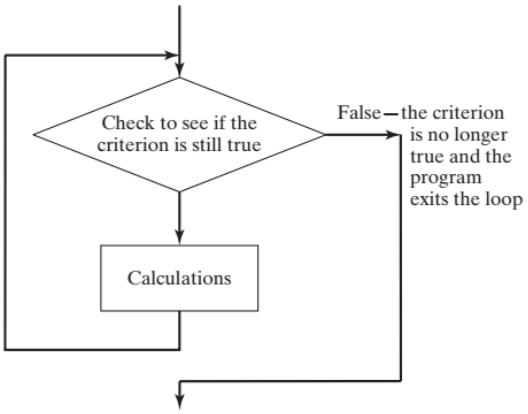
%display the matrix
disp(t)
```

The `while` loop

- A block of statements that are repeated indefinitely as long as some condition is satisfied.

```
while expression
   ...
   ...
   ...
end
```

Code block



The `while` loop

- The following code

```
k = 0;
while k<3
   k = k+1
end
```

- Should return

```
k =
   1
k =
   2
k =
   3
```

- The following code

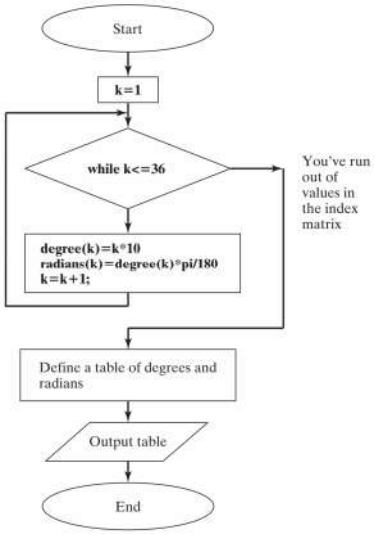
```
a = 0;
while(a<10)
 a = a + 3
end;
```

- Should return

```
a =
   3
a =
   6
a =
   9
a =
   12
```

The while loop

- Write a program that converts angle values from degrees to radians, from 0 to 360 degrees, in increments of 10 degrees



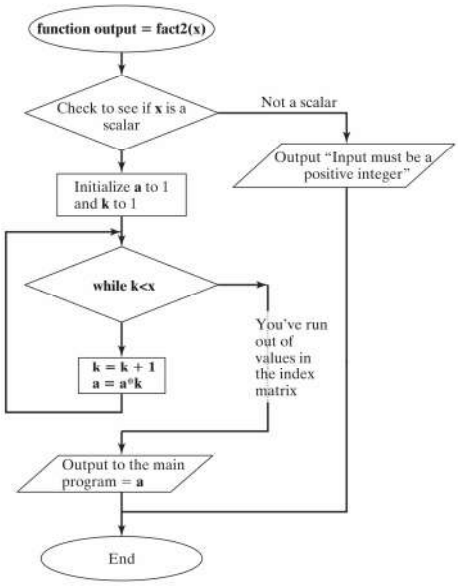

```
%Script to convert degrees to radians
%Create a matrix of input angles in degrees %on
row1 and convert them into radians on row 2

%Use a while loop for the calculations
%initialize the counter
k = 1;
%run the while loop to do the conversions
while k<=36
    degree(k) = k*10;
    radians(k) = degree(k)*pi/180;
    k = k+1;
end

%Create a matrix of results
t = [deg;rad]

%display the matrix
disp(t)
```


The while loop




```
function output = fact(x)
%This function accepts a scalar input and
%calculates its factorial

%Check to confirm that x is a positive integer
if(x<0)
   output = 'Input must be a positive integer'
else
% initialize a
a = 1;
%Initialize the counter
k=1
%run the while loop to calculate the factorial
while k <= x
  a = a*k;
  k = k + 1;
end
output = a
```

The `while` loop

```
%A script that asks for a scalar input and calculates its factorial

%Ask for input
x = input('Enter a positive value of x')

%Check to confirm that x is a positive integer
while(x<0)
   disp = ('Input must be a positive integer')
   x = input('Enter a positive value of x')
end

% initialize a
a = 1;
%Initialize the counter
k=1
%run the while loop to calculate the factorial
while k <= x
  a = a*k;
  k = k + 1;
end
output = a
```


The `break` and `continue` Statements

- Are used in the control of loops.
- The break statement terminates the execution of a loop when a condition is met.
- The continue statement terminates the current pass through the loop and returns control to the top of the loop

The `break` Statements

- A code that takes input vectors and determines the first element whose square is 25.

```
%A script that determines the first element in a given vector whose product
%is 25 when squared.

%say given vector is a = [2 5 3 9 7]

%Initialize count
Count = 1

%Check to confirm that x is a positive integer
for a = [2 5 3 9 7]
   aa = a^2
   count = count + 1
     if aa == 25
        break
     end
end
disp('The index of the first element that gives 25 when squared is')
disp(count)
```


The `continue` Statement

```
for ii = 1:5
  if ii = = 3
   continue;
  end
end
disp(['End of loop'])
```

When this program is executed, the output is:

```
ii = 1
ii = 2
ii = 4
ii = 5
End of loop
```